

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: Abhishek Yandra

Roll No.: A24126552127

Date: 17-08-2026

1. TITLE

Develop a Dynamic Webpage Using HTML5, CSS3, and JavaScript

2. AIM

To design and develop a dynamic webpage using HTML5, CSS3, and JavaScript that implements a Counter App, demonstrating DOM manipulation, external file linking, event handling via onclick attributes, and CSS Flexbox-based layout and button styling.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the structure of a multi-file web project with separate HTML, CSS, and JavaScript files.
 - To link an external CSS stylesheet and an external JavaScript file to an HTML document.
 - To use the `<span>` element with an id to display a dynamically updated value on the page.
 - To implement onclick event handlers on buttons to call JavaScript functions.
 - To manipulate the DOM using `document.getElementById()` and `.innerText` to update the counter display.
 - To use CSS Flexbox properties to centre page content both horizontally and vertically.
 - To apply button styling including background colour, border-radius, padding, and hover effects using CSS pseudo-classes.
-

4. THEORY

Dynamic Web Page — a web page whose content changes in response to user interactions without reloading, achieved by using JavaScript to read and modify HTML elements through the Document Object Model (DOM).

DOM Manipulation — the process of accessing and modifying HTML elements from JavaScript at runtime. `document.getElementById(id)` returns the element with the matching id attribute, and properties such as `innerText` allow its content to be read or updated.

Event Handling — `onclick` is an HTML attribute that binds a JavaScript function call to a button click event. When the button is clicked, the browser calls the named function defined in the linked script file.

External Files — separating HTML structure, CSS presentation, and JavaScript behaviour into distinct files improves maintainability. The HTML links them using `<link rel="stylesheet">` for CSS and `<script src="...">` for JavaScript.

CSS Flexbox — a layout model that aligns and distributes space among items in a container. Setting `display: flex` with `justify-content: center` and `align-items: center` on the body centres the main container in the viewport.

CSS Pseudo-classes — selectors that apply styles based on element state. The `button:hover` rule changes the button background when the pointer is over it, providing interactive visual feedback without JavaScript.

5. ALGORITHM / PROCEDURE

- Create three files in the same folder: `Index.html`, `style.css`, and `script.js`.
- In `Index.html`, add the DOCTYPE declaration, `<html>`, `<head>`, and `<body>` elements.
- Inside `<head>`, link `style.css` using `<link rel="stylesheet" href="style.css">`.
- In `<body>`, add a `<div class="main-container">` containing an `<h1>` heading and a nested `<div>` for the Counter App.
- Inside the Counter App `<div>`, add an `<h2>`, a `<p>` with a `<span id="count-number">0</span>` to display the count, and three `<button>` elements with `onclick` attributes calling `increaseCount()`, `decreaseCount()`, and `resetCount()`.
- Add `<script src="script.js"></script>` just before the closing `</div>` of the main container.
- In `style.css`, set body to `display: flex` with `justify-content: center` and `align-items: center` to centre the layout.
- Style `.main-container` with `width: 60%` and `margin: 0 auto`, and `h1` with a dark colour.
- Style the button selector with `background-color`, `color`, `border`, `padding`, `margin`, `cursor`, and `border-radius` properties.
- Add a `button:hover` rule to darken the background colour on mouse-over.
- In `script.js`, declare `let count = 0` as the counter variable.
- Define `increaseCount()`: increment count by 1 and update the span `innerText`.
- Define `decreaseCount()`: decrement count by 1 and update the span `innerText`.
- Define `resetCount()`: set count to 0 and update the span `innerText`.
- Open `Index.html` in a browser, test all three buttons, and record expected vs. actual output for each test case.

6. PROGRAM

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Dynamic Webpage</title>
  <!-- Linking CSS file -->
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="main-container">
    <h1>Dynamic Web Page</h1>
    <!-- Dynamic Counter -->
    <div>
      <h2>Counter App</h2>
      <p>Current Count: <span id="count-number">0</span></p>
      <button onclick="increaseCount()">Increase (+1)</button>
      <button onclick="decreaseCount()">Decrease (-1)</button>
      <button onclick="resetCount()">Reset</button>
    </div>
    <script src="script.js"></script>
  </div>
</body>
</html>
```

style.css

```
h1 {
  color: #333;
}
.main-container {
  width: 60%;
  margin: 0 auto;
}
```

```
body {
  font-family: Arial, sans-serif;
  background-color: #f0f8ff;
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  margin: 0;
  text-align: center;
}
button {
  background-color: #008CBA;
  color: white;
  border: none;
  padding: 8px 15px;
  margin: 5px;
  cursor: pointer;
  border-radius: 4px;
}
button:hover {
  background-color: #005f73;
}
```

script.js

```
// Variable for counter
let count = 0;

// 1. Counter Functions using document.getElementById
function increaseCount() {
  count = count + 1;
  document.getElementById("count-number").innerText = count;
}

function decreaseCount() {
  count = count - 1;
```

```

    document.getElementById("count-number").innerText = count;
}

function resetCount() {
    count = 0;
    document.getElementById("count-number").innerText = count;
}

```

7. CODE EXPLANATION

Code	Explanation
<code><link rel="stylesheet" href="style.css"></code>	Links the external CSS file style.css to the HTML page so all styling rules are applied.
<code><script src="script.js"></script></code>	Links the external JavaScript file script.js, placed just before the closing <code></div></code> so the DOM elements are available when the script runs.
<code>0</code>	A span element that displays the current counter value. Its id allows JavaScript to target and update it directly.
<code>onclick="increaseCount()"</code>	An inline event handler that calls increaseCount() in script.js each time the Increase button is clicked.
<code>let count = 0</code>	Declares and initialises the counter variable to 0. This variable persists across function calls and tracks the current count.
<code>count = count + 1</code>	Increments the counter by 1 inside increaseCount(). The updated value is then written back to the DOM.
<code>count = count - 1</code>	Decrements the counter by 1 inside decreaseCount(). The updated value is then written back to the DOM.
<code>count = 0 (resetCount)</code>	Resets the counter variable to 0 inside resetCount(), returning the display to its initial state.
<code>document.getElementById("count-number")</code>	DOM method that locates the element with id="count-number" so its text content can be updated.
<code>.innerText = count</code>	Sets the visible text of the span element to the current value of count, updating the display immediately.
<code>display: flex; justify-content: center</code>	CSS Flexbox properties that centre the main container both horizontally and vertically within the viewport.
<code>button:hover { background-color: #005f73; }</code>	CSS pseudo-class rule that darkens the button background when the pointer hovers over it, providing visual feedback.

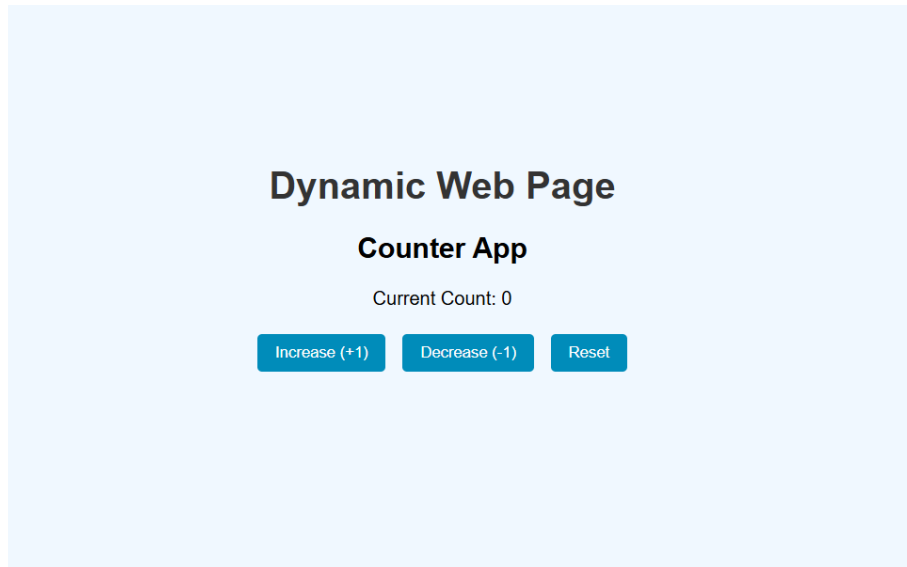
8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Load Index.html in a browser	Page renders with heading "Dynamic Web Page", "Counter App" subheading, count showing 0, and three buttons	Page rendered correctly with all elements visible	Pass
TC-02	Click Increase (+1) once	Count display updates from 0 to 1	Count displayed as 1	Pass
TC-03	Click Increase (+1) five times	Count display updates to 5	Count displayed as 5	Pass
TC-04	Click Decrease (-1) once	Count display decrements by 1	Count decremented correctly	Pass
TC-05	Click Decrease (-1) when count is 0	Count display updates to -1 (no lower bound enforced)	Count displayed as -1	Pass
TC-06	Click Reset after several increments	Count display resets to 0	Count displayed as 0	Pass
TC-07	Hover over a button	Button background changes to darker colour (#005f73)	Hover colour applied correctly	Pass
TC-08	Inspect page layout	Main container centred horizontally and vertically on the page with light blue background	Flexbox centering applied correctly	Pass
TC-09	Inspect button styling	Buttons display with blue background, white text, rounded corners, and spacing	Button styles rendered as defined in style.css	Pass
TC-10	Alternate Increase and Decrease clicks	Count value reflects the net result of all clicks correctly	Counter updated accurately for each click	Pass

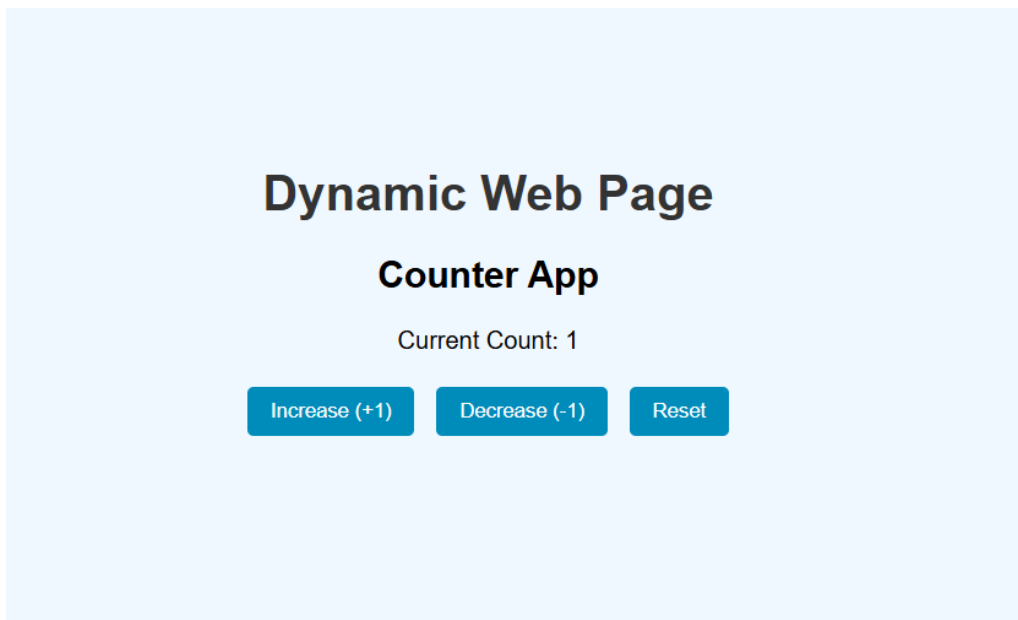
9. OUTPUT

Output 1: Initial Page Load

The browser displays the "Dynamic Web Page" heading centred on a light blue background. The Counter App section shows "Current Count: 0" and three styled blue buttons — Increase (+1), Decrease (-1), and



Output 2: Increase Button(+1)



Output 3: Increase count 5 times

Dynamic Web Page

Counter App

Current Count: 5

Increase (+1)

Decrease (-1)

Reset

Output 4: Decrease count(-1)

Dynamic Web Page

Counter App

Current Count: 4

Increase (+1)

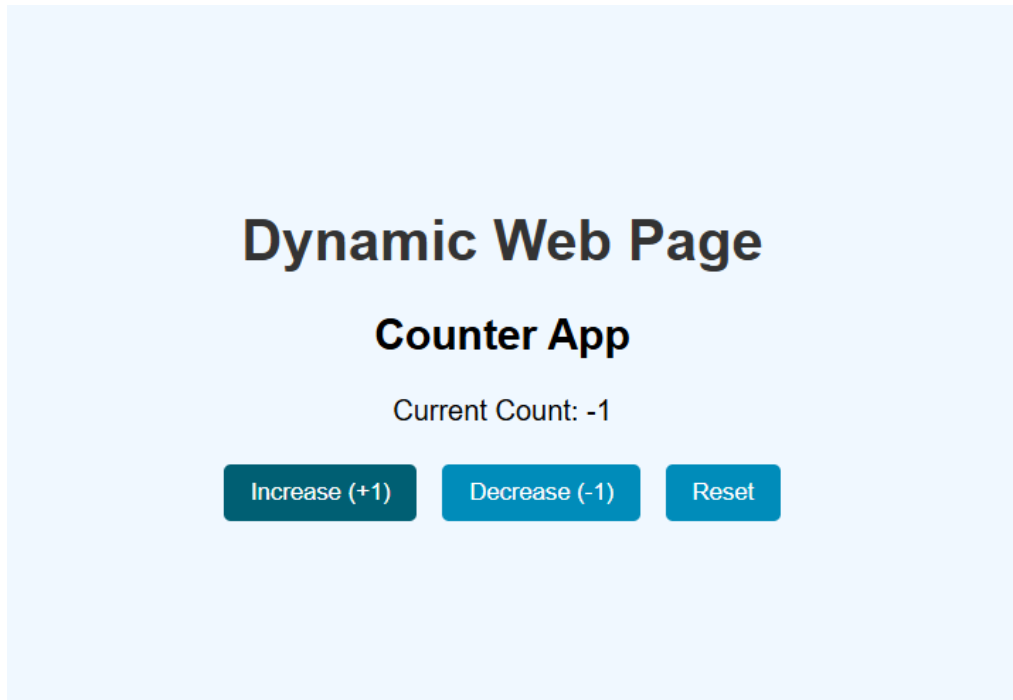
Decrease (-1)

Reset

Increase (+1) button increments the displayed count by 1. The span element updates immediately without a page reload, demonstrating live DOM manipulation via innerText.

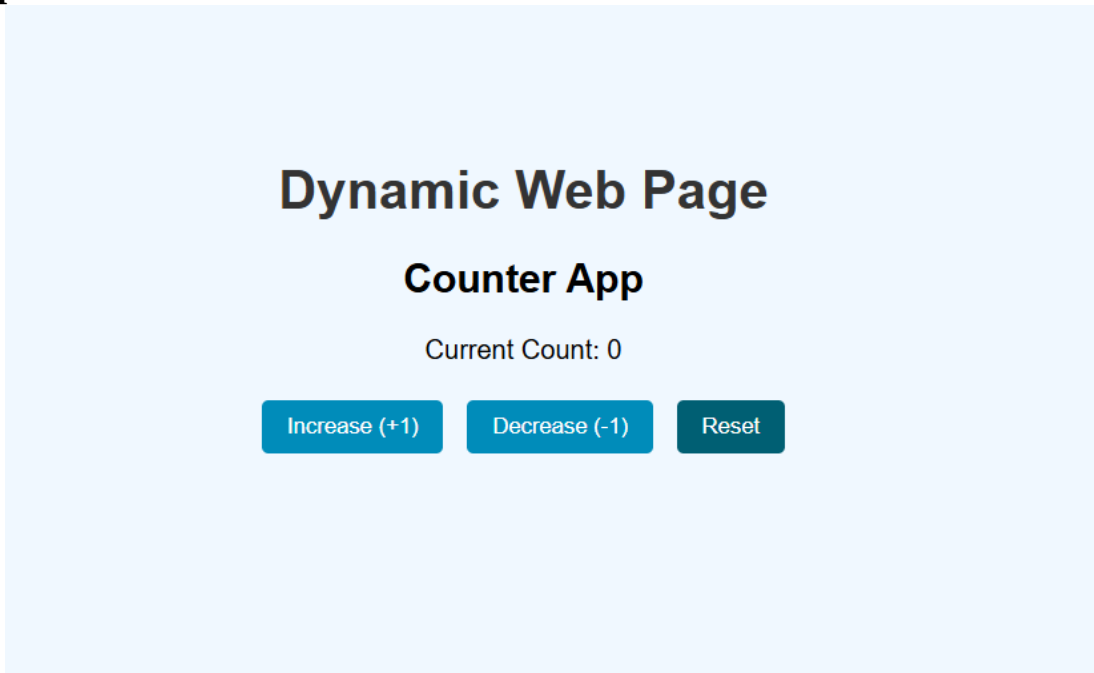
Output 5: Decrease to -1

Clicking Decrease (-1) decrements the count by 1 on each click, including below zero and updates the display.



Output 6: Reset Button

Clicking
Reset
sets
the
count
back
to 0



Clicking

instantly, confirming that `resetCount()` correctly reassigns the variable and updates the display.

10. RESULT

Thus, a dynamic webpage was successfully designed and developed using HTML5, CSS3, and JavaScript across three linked files. The page demonstrates external file linking, DOM manipulation using `document.getElementById()` and `innerText`, onclick event handling, CSS Flexbox-based centred layout, and interactive button styling with hover effects. All ten test cases passed successfully, confirming correct counter behaviour, layout, and styling.